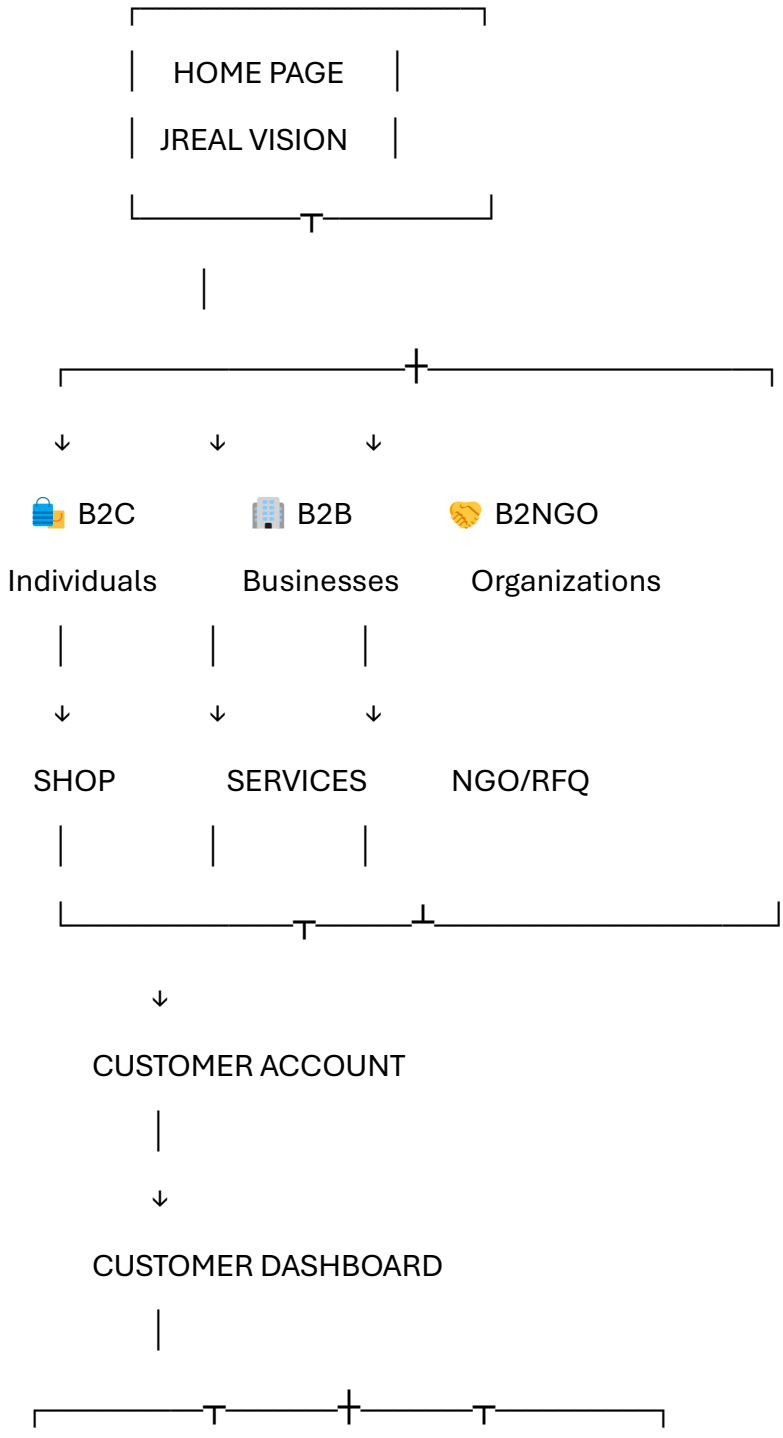
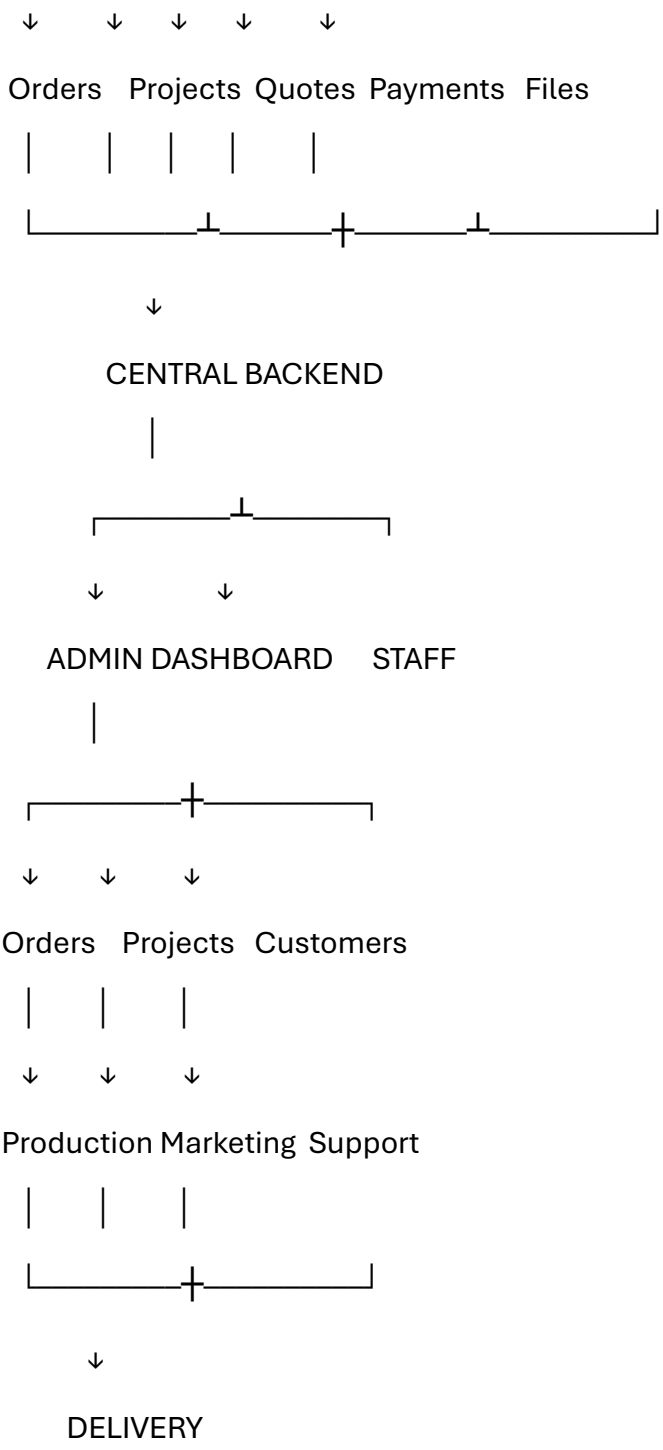
Yes. The best way to explain this to your developer is to give them a **complete system logic document**: how a visitor enters from the homepage, how they become a customer, how orders/projects are created, and how everything reaches the admin/backend.

Think of the website as **one front door with different paths**, but **one central backend**.

JREAL VISION — COMPLETE WEBSITE LOGIC

1. The whole system





---

2. HOME PAGE = THE TRAFFIC CONTROLLER

The homepage should **not** contain every feature.

Its job is to ask:

“What do you need?”

Main buttons

[SHOP PRODUCTS]

[DESIGN & PRINT]

[BUSINESS SOLUTIONS]

[NGO & ORGANIZATION]

[DIGITAL MARKETING]

[PHOTOGRAPHY]

[VIDEOGRAPHY]

[REQUEST A QUOTE]

Each button sends the visitor into the correct workflow.

---

3. B2C FLOW

Someone wants a T-shirt.

HOME

↓

SHOP

↓

T-SHIRTS

↓

SELECT PRODUCT

↓

SELECT SIZE / COLOR / QUANTITY

↓

UPLOAD DESIGN

↓

PRICE CALCULATED

↓

ADD TO CART

↓

CHECKOUT

↓

LOGIN / CREATE ACCOUNT / GUEST

↓

PAYMENT

↓

ORDER CREATED

↓

CUSTOMER DASHBOARD

↓

ORDER TRACKING

↓

DELIVERY

Backend

When they click **Place Order**, the backend creates:

Customer

↓

Order

↓

Order Items

↓

Payment

↓

Production

↓

Delivery

---

4. B2B FLOW

Suppose a restaurant wants:

- Logo
- Menu
- 50 T-shirts
- 500 business cards
- Product photography
- Social media management

Don't make them create six unrelated orders.

They click:

BUSINESS SOLUTIONS

Then:

Business Solutions

↓

Create Business Project

↓

Select Services

↓

Logo

Printing

Photography

Marketing

↓

Describe Requirements

↓

Upload Files

↓

REQUEST QUOTE

↓

JREAL ADMIN

↓

Review Requirements

↓

Create Quotation

↓

Customer Dashboard

↓

Customer Reviews Quote

↓

Accept / Negotiate / Reject

↓

AGREED PRICE

↓

Deposit Payment

↓

PROJECT STARTED

---

5. B2NGO FLOW

This should be slightly different.

NGO visits:

NGO & ORGANIZATION

They see:

Creative, Printing, Media & Marketing Solutions for Organizations

Then:

NGO PAGE

↓

REQUEST NGO QUOTE

↓

Organization Account

↓

Organization Information

↓

Project Information

↓

Upload RFQ / TOR / BOQ / Specifications

↓

Submit Request

↓

BACKEND

↓

ADMIN REVIEWS

↓

Quotation / Proposal

↓

Customer receives notification

↓

Review quotation

↓

Negotiate if necessary

↓

Approve

↓

Purchase Order / Agreement

↓

Deposit / Payment

↓

Production

↓

Quality Check

↓

Delivery

↓

Project Completed

---

6. ONE ACCOUNT FOR EVERYTHING

This is very important for your developer.

Do **not** create separate systems for B2C, B2B and NGO.

Use:

USER

│

├— Personal Profile

│

├— Business Profile (optional)

│

├— Organization Profile (optional)

│

├── Orders

├── Projects

├── Quotes

├── Payments

├── Files

├── Messages

└── Bookings

A person can therefore start as an individual and later create a business/organization profile.

---

7. CUSTOMER DASHBOARD

The customer should see only what is relevant.

CUSTOMER DASHBOARD

Overview

│

├── My Orders

├── My Projects

├── My Quotes

├── My Payments

├── My Files

├── My Bookings

├── Messages

├— Notifications

└— Profile

Don't expose complicated production functions to customers.

---

8. QUOTATION SYSTEM

This is one of the most important backend features.

There are two types.

Fixed-price

Example:

Business Card — 500 pcs — 35,000 RWF

Customer:

Add to Cart → Checkout

Custom quotation

Example:

NGO wants 5,000 training manuals.

Customer:

Request Quote

Backend creates:

QUOTE #QT-1025

Customer

Project

Services

Quantity

Specifications

Estimated price

Discount

Final price

Validity date

Terms

Status

---

9. NEGOTIATION LOGIC

The quote should have:

DRAFT

↓

SENT TO CUSTOMER

↓

CUSTOMER ACTION

├— ACCEPT

├— REJECT

└— NEGOTIATE

If negotiate:

JREAL: 600,000

↓

Customer: 500,000

↓

JREAL: 550,000

↓

Customer: ACCEPT

↓

FINAL PRICE = 550,000

Then the backend **locks the agreed price**.

The customer should not be able to change it afterward.

---

10. PROJECT MANAGEMENT

Every large service should become a **Project**.

For example:

PROJECT #PRJ-001

ABC Restaurant Branding

Services:

✓ Logo

✓ Business Cards

✓ T-Shirts

✓ Photography

✓ Video

✓ Marketing

Status:

Quotation

↓

Approved

↓

Deposit Paid

↓

Design

↓

Production

↓

Media

↓

Marketing

↓

Completed

This allows one customer to have a complete business project.

---

11. FILE MANAGEMENT

Every uploaded file must belong to something.

FILE

↓

CUSTOMER

↓

PROJECT

↓

SERVICE

Example:

logo.png

↓

ABC Restaurant

↓

Branding Project

↓

Logo Design

Don't put all customer files into one public folder.

The backend should control who can access each file.

---

12. PAYMENT LOGIC

Payment should connect to:

Order OR Quote/Project

Example:

QUOTE

600,000 RWF

↓

Customer accepts

↓

Deposit required

300,000 RWF

↓

Payment

↓

PAYMENT CONFIRMED

↓

PROJECT ACTIVE

Then:

Total: 600,000

Paid: 300,000

Balance: 300,000

---

13. ADMIN BACKEND

Your developer should build a completely separate **Admin Dashboard**.

ADMIN DASHBOARD

Overview

│

├— Customers

├— Orders

├— Projects

├— Quotations

├— Negotiations

├— Payments

├— Products

├— Services

├— Files

├— Bookings

├— Marketing Campaigns

├— Production

├— Delivery

├— Messages

├— Reviews

└— Reports

---

14. ADMIN ORDER FLOW

When someone orders a banner:

CUSTOMER

↓

ORDER CREATED

↓

PAYMENT CHECK

↓

ARTWORK CHECK

↓

PRODUCTION

↓

QUALITY CONTROL

↓

READY

↓

DELIVERY / PICKUP

↓

DELIVERED

↓

REVIEW

Each status change should automatically notify the customer.

---

15. DIGITAL MARKETING BACKEND

This should be a separate module connected to the customer/project.

CLIENT

↓

MARKETING PROJECT

↓

CAMPAIGN

↓

CONTENT PLAN

↓

CONTENT CREATION

↓

CLIENT APPROVAL

↓

PUBLISH

↓

CAMPAIGN RESULTS

↓

MONTHLY REPORT

Customer dashboard:

Your September Marketing Campaign

- 20 posts
- 8 reels
- 4 campaigns
- Content calendar
- Results/report

---

16. PHOTOGRAPHY / VIDEOGRAPHY

These should use a **booking system**.

SERVICE

↓

BOOKING REQUEST

↓

DATE / TIME / LOCATION

↓

ADMIN CONFIRMATION

↓

QUOTE

↓

PAYMENT

↓

BOOKING CONFIRMED

↓

SESSION

↓

FILES DELIVERED

↓

COMPLETED

---

17. DATABASE STRUCTURE

Give your developer something similar to this:

USERS

├— user_id

├— name

├— email

├— phone

├— password_hash

└— role

PROFILES

├— profile_id

├— user_id

├— type

└— information

ORGANIZATIONS

├— organization_id

├── owner_id

├── name

└── organization_type

PRODUCTS

├── product_id

├── name

├── category

├── price

├── pricing_type

└── stock

ORDERS

├── order_id

├── customer_id

├── total

├── status

└── payment_status

ORDER_ITEMS

├── order_item_id

├── order_id

├── product_id

├── quantity

└── price

PROJECTS

├── project_id

├── customer_id

├── project_type

├── status

└── total

QUOTES

├── quote_id

├── project_id

├── amount

├── discount

├── final_price

├── status

└── valid_until

NEGOTIATIONS

├── negotiation_id

├── quote_id

├── user_id

├── offer_amount

├── message

└── status

PAYMENTS

├── payment_id

├── customer_id

├── order_id

├── quote_id

├── amount

├── method

└── status

FILES

├── file_id

├── customer_id

├── project_id

├── filename

└── secure_url

MESSAGES

├── message_id

├── project_id

├── sender_id

├── message

└── timestamp

BOOKINGS

├— booking_id

├— customer_id

├— service_id

├— date

├— location

└— status

NOTIFICATIONS

├— notification_id

├— customer_id

├— title

├— message

└— read_status

---

18. The most important backend principle

Tell your developer:

Everything must have an ID and relationship.

For example:

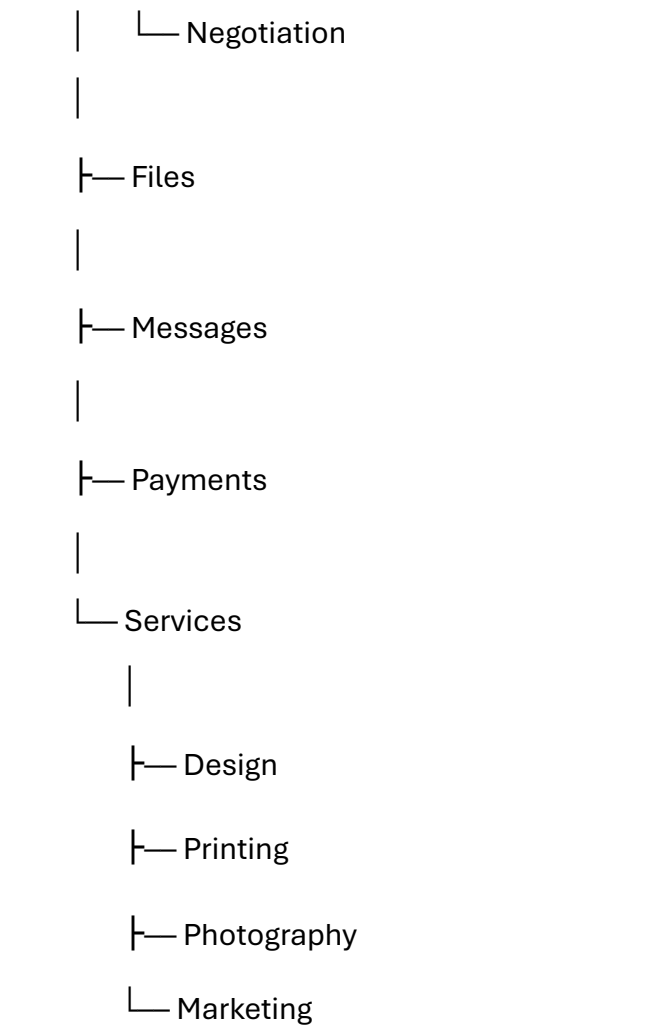
Customer

  │

  └— Project #PRJ001

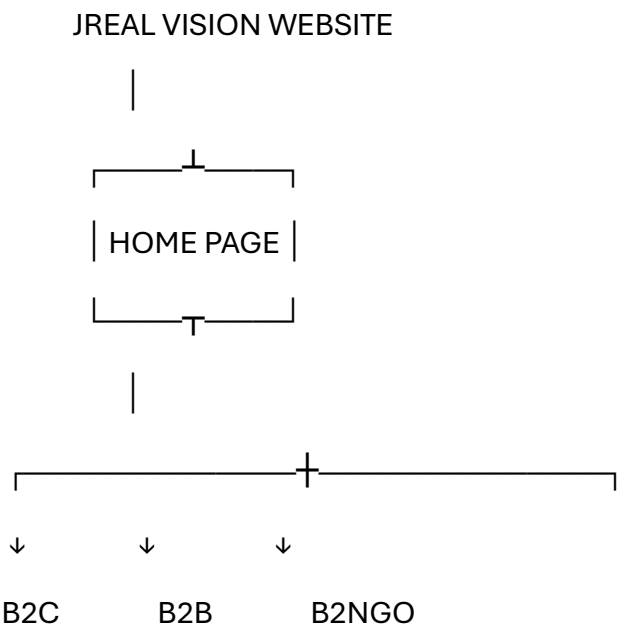
      │

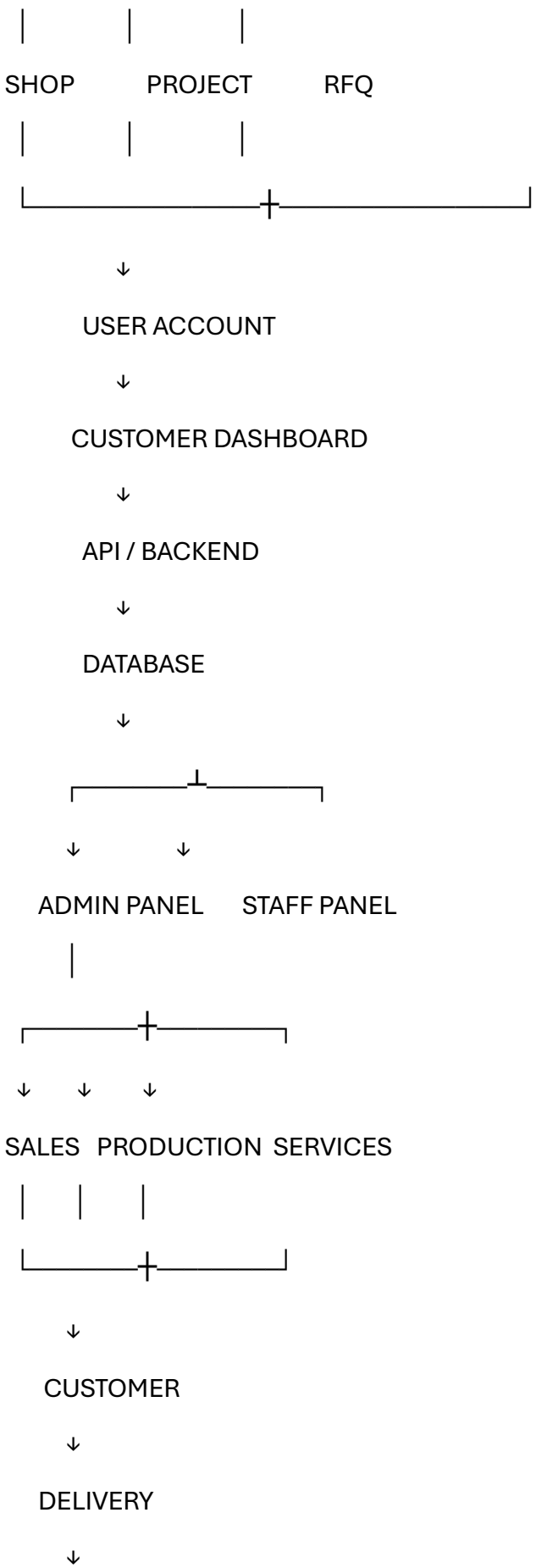
      ├— Quote #QT001



This prevents the system from becoming messy later.

Final architecture




SHOP
PROJECT
RFQ
USER ACCOUNT
CUSTOMER DASHBOARD
API / BACKEND
DATABASE
ADMIN PANEL
STAFF PANEL
SALES
PRODUCTION
SERVICES
CUSTOMER
DELIVERY

REVIEW

This is the architecture I would give your developer before coding. It keeps B2C, B2B and B2NGO under one platform while allowing each customer type to have a different buying/request process.